location in the cache where a particular memory address can be mapped. Consider a trivial example of a cache with 16 entries. In this case, there are 16 sets, each containing only one cache entry, numbered 0 to 15. A given memory address is mapped, using a hash function address % 16, to a set in the cache. Since there is only one cache entry in each set, if the application accesses addresses in the order 0, 16, 32, 0, 8, 16 and 32, each will get mapped to set 0. Since set 0 has only one entry, when Address 16 is accessed, it has to evict the data corresponding to Address 0. Similarly, when Address 32 is accessed, it will again evict the data corresponding to Address 0, since they all get mapped to the same cache set. Now, if Address 16 is accessed again, it will have a miss, since Address 32 has replaced it in the cache. Therefore, a direct-mapped cache can lead to conflicts between different cache entries. Cache misses that occur due to multiple memory addresses mapping to the same cache entry are known as conflict misses.

Set-associative caches were introduced to reduce conflict misses. Consider the above example with a 3-way set-associative cache. In this case, each cache set contains three cache entries. Therefore, all three addresses (0, 16 and 32) can be present simultaneously in the cache, without having to evict each other, resulting in zero conflict misses. Note, however, that set-associative caches can lead to an increase in cache look-up time—since there are *k* entries where a particular memory location can be mapped, all *k* entries need to be checked to see if the data is present in the cache or not, leading to a possible increase in cache access time.

Q What is the difference between little-endian architecture and big-endian architecture?

Little-endian and big-endian refer to the order in which the data bytes are stored in memory. Little-endian stands for 'Little End In'. In this architecture, the little end is stored first, i.e., the memory address contains the least significant byte stored first. For example, a data value *0x12345678* stored at location X would be stored as *(0x78 0x56 0x34 0x12)*, with the memory location *X* containing *0x78*, the memory location *X+1* holding *0x56*, the memory location *X+2* storing *0x34*, and the memory location *X+3* having *0x12*. Intel x86 processors follow the little-endian convention.

In big-endian architecture, which stands for 'Big End In', the memory address contains the most significant byte stored first. For example, a data value *0x12345678* stored at location X would be stored as *(0x12 0x34 0x56 0x78)*, with the memory location *X* containing *0x12*, *X+1* storing *0x34*, and so on. Both Motorola and HP's PA-RISC/IA-64 processors follow the big-endian convention.

Q What is the difference between symmetric multiprocessor (SMP) systems and chip multiprocessor systems?

An SMP system consists of two or more separate processors connected, using a common bus, switch hub, or network, to shared memory and I/O devices. The overall system is usually physically smaller, and uses less power than an equivalent set of uni-processor systems, because physically large components such as memory, hard drives, and power supplies can be shared by some or all of the processors.

So, when faced with the power wall and memory wall, chip designers decided to apply the same principles of scale out from SMP systems, down to the chip level. They introduced chip multiprocessor systems, wherein two or more processor cores are placed on a single chip, and share certain processor resources such as cache.

Q What is the difference between write-allocate and write-no-allocate cache?

When a processor writes to a memory location, it can either first allocate an entry in the cache and write to the entry in the cache (known as write-allocate), or it can directly write to the main memory without allocating a cache entry (known as write-no-allocate). In case of a write-allocate cache, the processor has a choice of when to update the main memory with the latest value from the cache—for that memory location.

My 'must read book' for this month

This month's book suggestion comes from S Priya. Her recommendation is, 'Programming Pearls' by Jon Bentley. While this book is not meant for a first course in programming, it is a great find for those trying to sharpen their skills in design and coding. The book provides multiple solutions to each of the programming problems, by discussing the pros and cons of each approach. This makes it easier for the reader to appreciate how to apply the techniques mentioned in the book. So do make time to read it.

If you have a favourite programming book that you think is a must-read for every programmer, please do send me a note with the book's name, and a short note on why you think it is useful. I will mention it in this column. This will help readers who want to improve their coding skills.

Please do send me your answers to this month's questions. If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming, and wish you the very best! END

About the author:

Sandya Mannarswamy. The author is a specialist in compiler optimisation and works at Hewlett-Packard India. She has a number of publications and patents to her credit, and her areas of interest include virtualisation technologies and software development tools. *[If you are preparing for computer science programming interviews, you may find it useful to visit Sandya's programming interviews discussion group 'Computer Science Interview Training (India)' on LinkedIn (www.linkedin.com).]*